Agile Software Development

Prepared By

SREEDEV C S

November 13, 2026

Contents

1 Theoretical Background **1**

1.1 Introduction to Agile Software Development 1

1.1.1 Agile Manifesto 1

1.2 Agile vs Traditional Software Development 2

1.3 Agile Methodologies and Frameworks 3

1.3.1 Scrum 3

1.3.2 Kanban 4

1.3.3 Extreme Programming (XP) 5

1.3.4 Lean Software Development 5

1.3.5 Feature-Driven Development (FDD) 5

1.4 Work Item Hierarchy in Agile 5

1.4.1 Epic 6

1.4.2 Feature 6

1.4.3 User Story 6

1.4.4 Task 6

1.4.5 Acceptance Criteria 7

1.5 Product Backlog and Sprint Backlog 8

1.5.1 Product Backlog 8

1.5.2 Sprint Backlog 8

1.5.3 Difference Between Product Backlog and Sprint Backlog 9

1.6 Sprint and Sprint Lifecycle 9

1.6.1 Estimation of Tasks 10

2 Practical Case Study: Amazon.in **11**

2.1 Introduction 11

2.2 Identification of Epic 12

2.3 Identification of Features 12

2.4 Identification of User Stories 13

2.4.1 Feature 1: Your Orders 13

2.4.2 Feature 2: Returns and Refunds 13

2.4.3 Feature 3: Manage Address 14

2.4.4 Feature 4: Manage Prime 14
2.5 Identification of Tasks 15
2.5.1 User Story 1: Track Packages 16
2.5.2 User Story 2: Edit or Cancel Orders 17
2.5.3 User Story 3: Return and Exchange Items 18
2.5.4 User Story 4: Print Labels 18
2.6 Summary of the Work Item Hierarchy 19

Chapter 1

Theoretical Background

1.1 Introduction to Agile Software Development

Agile is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, continuous feedback, and frequent delivery of working software [2]. Traditional software development approaches generally follow a sequential process in which requirements, design, implementation, testing, and deployment are performed in distinct phases. Agile, on the other hand, divides the development process into smaller iterations, allowing teams to develop, test, and deliver software incrementally. Agile encourages continuous interaction between developers, customers, and other stakeholders. This enables development teams to accommodate changing requirements and improve the product based on regular feedback. The Agile approach is particularly useful in projects where requirements may evolve during development.

1.1.1 Agile Manifesto

The Agile Manifesto, introduced in 2001, defines four fundamental values that guide Agile software development:

1. **Individuals and interactions over processes and tools:** Effective communication and collaboration among team members are prioritized.

2. **Working software over comprehensive documentation:** Delivering functional software is prioritized over producing extensive documentation.

3. **Customer collaboration over contract negotiation:** Continuous engagement with customers helps ensure that the developed product satisfies their expectations.

4. **Responding to change over following a plan:** Development teams should be able to adapt to changing requirements rather than rigidly following an initial plan.

These values do not eliminate the importance of documentation, planning, processes, or contracts. Instead, they emphasize the importance of the values on the left-hand side.

1.2 Agile vs Traditional Software Development

Traditional software development approaches, such as the Waterfall model, follow a sequential development process. Each phase is generally completed before the subsequent phase begins. Agile follows an iterative and incremental approach in which development activities are performed in smaller cycles. Each cycle produces a working increment of the software. The major differences between these approaches are presented in Table 1.2.

Aspect	**Traditional Approach**	**Agile Approach**
Development	Sequential	Iterative and incremental
Requirements	Generally defined upfront	Can evolve throughout development
Customer involvement	Mainly at defined stages	Continuous involvement
Testing	Often performed after development	Integrated throughout iterations
Delivery	Usually at the end of the project	Frequent incremental delivery
Handling changes	Changes may be expensive	Designed to accommodate changes

Table 1.1: The differences between Tranditional and Agile approaches

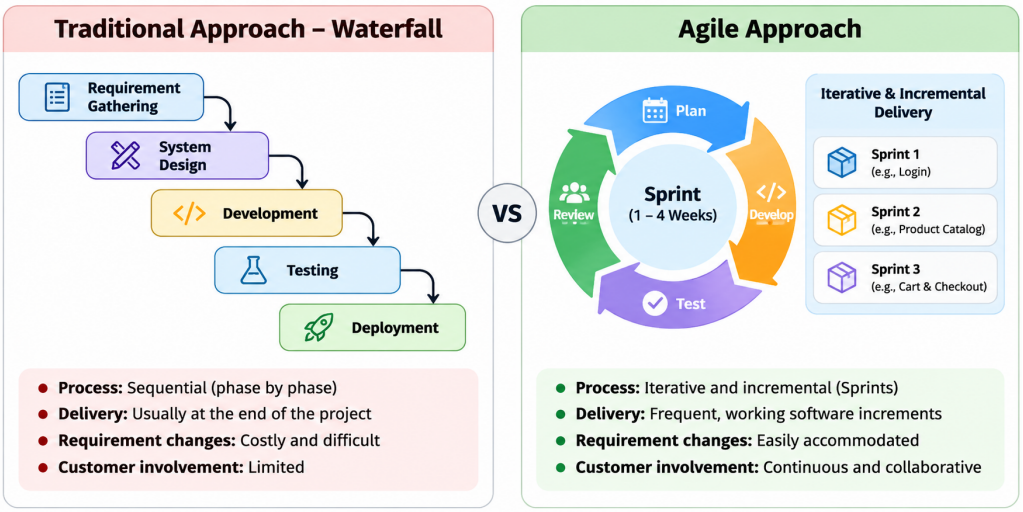


Figure 1.1: Comparison of Traditional and Agile Development

1.3 Agile Methodologies and Frameworks

Agile is an overarching software development approach rather than a single methodology. Several methodologies and frameworks implement Agile principles in different ways. Some of the commonly used Agile methodologies and frameworks are:

1.3.1 Scrum

Scrum is an Agile framework that organizes development into fixed-length iterations called Sprints [3]. Each Sprint produces a usable increment of the software. Scrum emphasizes collaboration, transparency, inspection, and continuous improvement.

The major components of Scrum include:

- **Product Backlog:** An ordered list of all requirements, features, enhancements, and other work items required for the product.
- **Sprint:** A fixed-length development cycle, generally lasting one to four weeks.
- **Sprint Backlog:** A collection of selected Product Backlog Items and the plan for delivering them during a particular Sprint.
- **Product Increment:** The usable and verified addition to the product developed during a Sprint.

Scrum defines three accountabilities:

- **Product Owner:** Responsible for maximizing product value and effectively managing the Product Backlog.
- **Scrum Master:** Responsible for establishing Scrum and helping the team improve its effectiveness.
- **Developers:** Responsible for creating a usable product increment during each Sprint.

Scrum also includes five formal events:

1. Sprint Planning
2. Daily Scrum
3. Sprint Review
4. Sprint Retrospective
5. The Sprint itself

1.3.2 Kanban

Kanban is an Agile workflow management method that focuses on visualizing work, limiting Work in Progress (WIP), and improving the continuous flow of activities.

A typical Kanban board consists of the following columns:

- To Do
- In Progress
- Testing
- Done

Each work item is represented by a card that moves across the board as it progresses. Unlike Scrum, Kanban does not require fixed-length Sprints. Work is continuously selected and delivered based on team capacity and priorities.

1.3.3 Extreme Programming (XP)

Extreme Programming is an Agile software development methodology that emphasizes engineering practices and code quality.

Its commonly used practices include:

- Pair Programming
- Test-Driven Development (TDD)
- Continuous Integration
- Continuous Refactoring
- Frequent Releases

1.3.4 Lean Software Development

Lean Software Development focuses on maximizing customer value while minimizing unnecessary activities and waste. Its major principles include eliminating waste, building quality into the product, delivering quickly, and continuously improving the development process.

1.3.5 Feature-Driven Development (FDD)

Feature-Driven Development organizes software development around small, client-valued features. The development process involves building an overall domain model, preparing a feature list, planning by feature, and designing and building individual features.

1.4 Work Item Hierarchy in Agile

In Agile software development, large requirements are divided into smaller and manageable work items. This decomposition helps development teams organize requirements, estimate effort, assign responsibilities, and monitor implementation progress.

A commonly used work item hierarchy is:

$$\textbf{Epic} \rightarrow \textbf{Feature} \rightarrow \textbf{User Story} \rightarrow \textbf{Task}$$

Each level represents a different degree of detail in the software development process.

1.4.1 Epic

An Epic represents a large body of work that encompasses multiple related functionalities. It generally cannot be completed within a single development task and is therefore divided into smaller work items. An Epic describes a broad business objective or a major functional area of the application. Epics help teams organize large requirements and track progress toward broader product objectives.

1.4.2 Feature

A Feature represents a specific functionality or capability provided by the software product. It describes what the system should offer to its users. An Epic can be divided into multiple Features, each addressing a particular aspect of the larger requirement.

1.4.3 User Story

A User Story describes a specific requirement from the perspective of an end user. It explains what the user wants to accomplish and why the functionality is useful. A commonly used format for writing User Stories is:

> As a `[type of user]`, I want `[some functionality]`, so that `[some benefit or purpose]`.

A User Story should focus on the user's requirement rather than describing the technical implementation.

1.4.4 Task

A Task represents an individual unit of work required to implement a User Story. Unlike a User Story, which describes the functionality from the user's perspective, a Task generally describes a specific development, testing, configuration, or implementation activity. Tasks are assigned to team members and can be estimated in terms of effort or duration. The tasks collectively contribute to the implementation of the User Story.

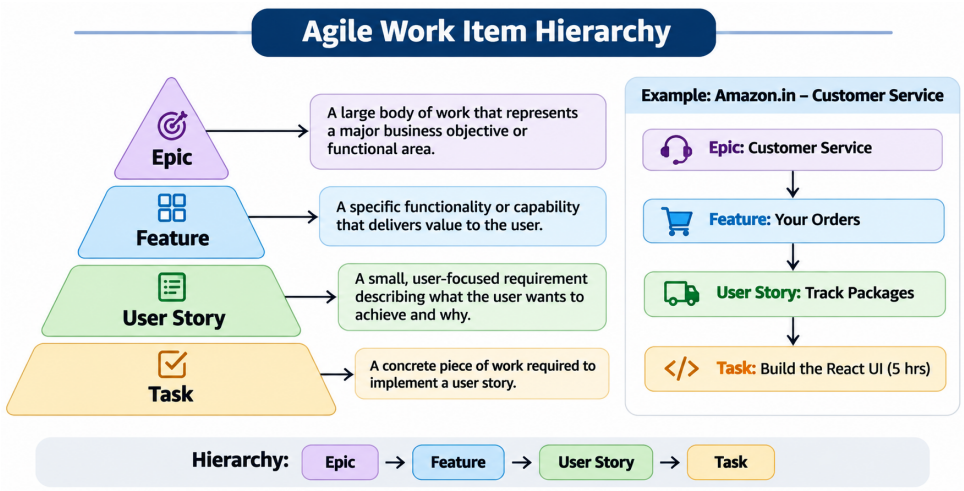


Figure 1.2: Hierarchical Decomposition of Agile Work Items

1.4.5 Acceptance Criteria

Acceptance Criteria are a set of conditions that a User Story must satisfy before it can be considered complete. They define the expected behavior of the functionality and provide a common understanding between the development team and the stakeholders.

For example, consider the User Story:

> `As a customer, I want to track my package so that I can know the current delivery status of my order.`

Possible Acceptance Criteria include:

1. The customer should be able to view their orders.
2. The customer should be able to select an order.
3. The system should display the current shipment status.
4. The system should display the estimated delivery date whenever available.
5. The customer should receive an appropriate message when tracking information is unavailable.

Acceptance Criteria are used to validate whether the implemented functionality satisfies the requirements of the User Story.

1.5 Product Backlog and Sprint Backlog

A Backlog is an organized collection of work items that need to be completed during software development.

1.5.1 Product Backlog

The Product Backlog is an ordered and continuously evolving list of all work required to improve or develop a product.

It may contain:

- Epics and Features
- User Stories
- Bug Fixes
- Technical Improvements
- Other Product Requirements

The Product Owner is accountable for effective Product Backlog management, including ensuring that backlog items are understood and appropriately ordered.

1.5.2 Sprint Backlog

The Sprint Backlog contains the work selected for a particular Sprint along with the plan for delivering it. During Sprint Planning, the team selects appropriate items from the Product Backlog based on priorities, capacity, and the Sprint Goal. The selected User Stories may subsequently be divided into smaller Tasks for implementation.

1.5.3 Difference Between Product Backlog and Sprint Backlog

Aspect	Product Backlog	Sprint Backlog
Scope	Entire product	Current Sprint
Contents	All identified product work	Selected work and implementation plan
Ownership	Product Owner is accountable for management	Developers manage the Sprint Backlog
Updates	Continuously refined	Updated as the team learns during the Sprint
Purpose	Organize and prioritize product work	Achieve the Sprint Goal

Table 1.2: Product Backlog vs Sprint Backlog

1.6 Sprint and Sprint Lifecycle

A Sprint is a fixed-length iteration in Scrum during which a usable product increment is created. Sprints are generally one month or less in duration.

Each Sprint follows a structured process:

1. **Sprint Planning:** The team determines the Sprint Goal and selects the work to be completed.
2. **Development:** Developers implement the selected User Stories and associated Tasks.
3. **Daily Scrum:** Developers inspect progress toward the Sprint Goal and adapt their plan.
4. **Sprint Review:** The team presents the completed increment to stakeholders and collects feedback.
5. **Sprint Retrospective:** The team identifies improvements to its working process.

The process is repeated for subsequent Sprints until the product objectives are achieved.

1.6.1 Estimation of Tasks

Development teams estimate the effort required to complete individual Tasks. Estimation can be expressed in hours, days, or other effort units.

For example, a User Story may involve the following Tasks:

- Frontend development: 5 hours
- Backend development: 6 hours
- Database design: 8 hours
- Unit testing: 4 hours

The estimated effort for these activities is 23 hours.

Task-level estimation helps teams understand the effort required for implementation and plan their Sprint workload. However, task estimates represent expected effort rather than guaranteed completion time.

Chapter 2

Practical Case Study: Amazon.in

2.1 Introduction

Amazon.in is an e-commerce platform that provides customers with various functionalities related to product discovery, purchasing, order management, payments, and customer service [1]. For this case study, the Customer Service section of Amazon.in is considered to demonstrate how a real-world application can be decomposed into different Agile work items.

The objective is to identify a major functional area as an Epic and subsequently break it down into Features, User Stories, and implementation Tasks. The selected Epic is `Customer Service`, which includes functionalities related to order management, returns, refunds, and customer account management.

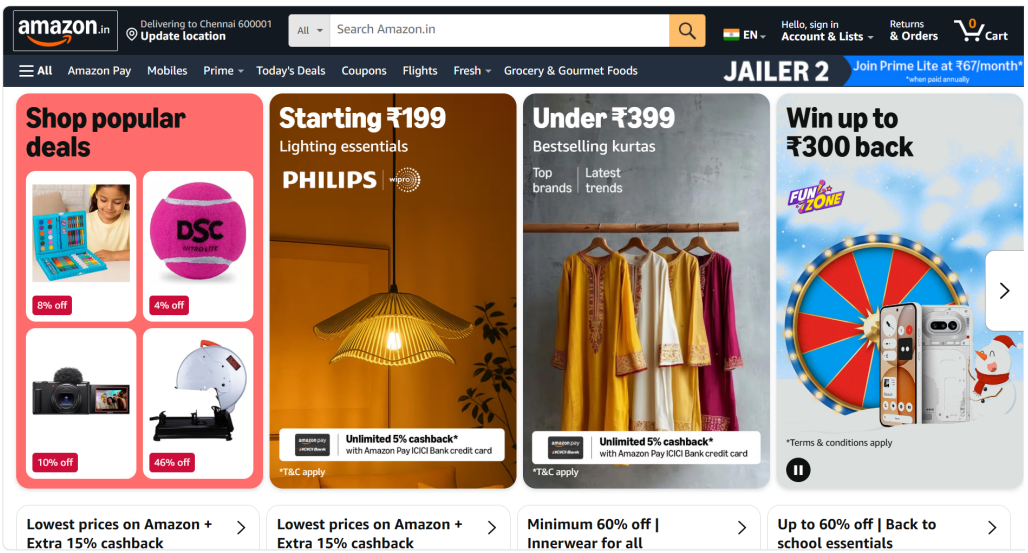


Figure 2.1: Amazon.in Homepage

2.2 Identification of Epic

An Epic represents a large body of related functionality within the application. For this case study, the selected Epic is:

Epic: Customer Service
Provides customers with functionalities to manage their orders, track deliveries, initiate returns, request refunds, and manage account-related information.

The Customer Service Epic encompasses multiple functionalities that collectively improve the customer's ability to manage their interactions with the e-commerce platform. These functionalities are further divided into individual Features.

2.3 Identification of Features

The Customer Service Epic can be decomposed into multiple Features, each representing a distinct area of functionality.

The following Features are considered for this case study:

1. **Your Orders:** Provides customers with functionalities to view their orders, track packages, and modify or cancel eligible orders.
2. **Returns and Refunds:** Enables customers to initiate product returns or exchanges, monitor refund progress, and obtain return labels where applicable.
3. **Manage Address:** Allows customers to add, edit, delete, and manage their saved delivery addresses.
4. **Manage Prime:** Provides customers with functionalities related to their Prime membership, including membership details and subscription management.

These Features represent different functional areas within the Customer Service Epic.

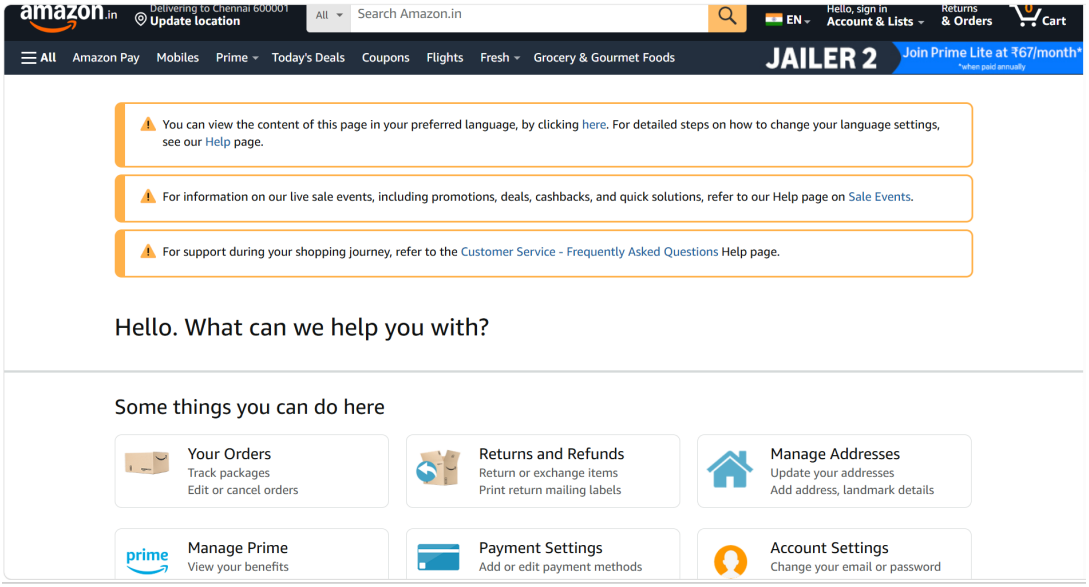


Figure 2.2: Customer Service Functionalities on Amazon.in

2.4 Identification of User Stories

Each Feature can be further divided into User Stories. A User Story describes an individual functionality from the perspective of the customer. The following User Stories are identified for the selected Features.

2.4.1 Feature 1: Your Orders

This Feature focuses on enabling customers to view and manage their previously placed orders.

The identified User Stories are:

1. **Track Packages:** Enables customers to view the current status and estimated delivery details of their orders.
2. **Edit or Cancel Orders:** Enables customers to modify or cancel eligible orders based on their current processing and shipment status.

2.4.2 Feature 2: Returns and Refunds

This Feature focuses on helping customers return purchased products and obtain refunds or replacements.

The identified User Stories are:

1. **Return and Exchange Items:** Enables customers to initiate a return or exchange request for eligible products.
2. **Print Labels:** Enables customers to obtain and print the required return shipping label when applicable.

2.4.3 Feature 3: Manage Address

This Feature allows customers to manage their saved delivery addresses.

Possible User Stories include:

1. Add a new delivery address.
2. Edit an existing delivery address.
3. Delete a saved delivery address.
4. Set a default delivery address.

2.4.4 Feature 4: Manage Prime

This Feature allows customers to manage their Prime membership-related information.

Possible User Stories include:

1. View Prime membership details.
2. Manage membership renewal settings.
3. View membership benefits.
4. Manage membership subscription.

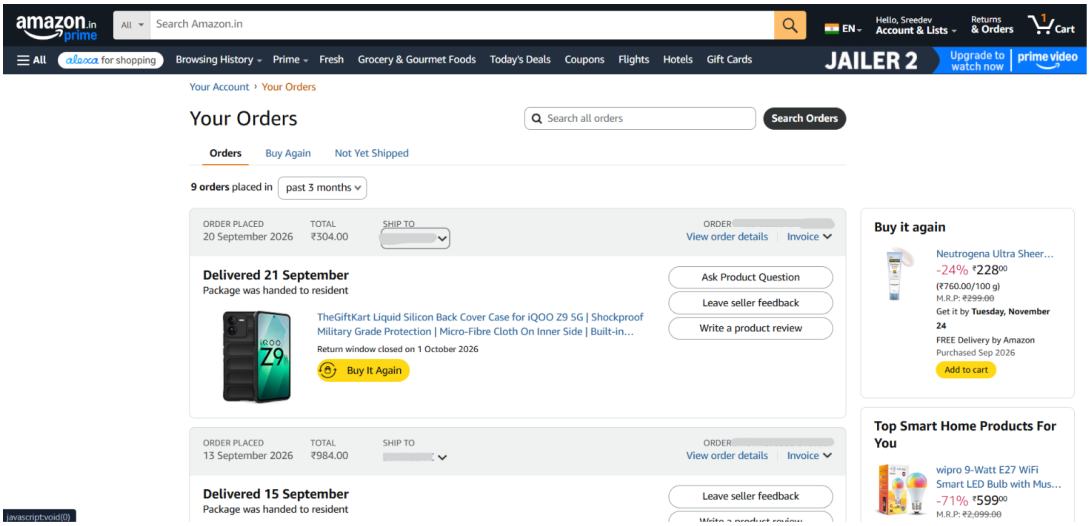


Figure 2.3: Your Orders Feature on Amazon.in

2.5 Identification of Tasks

Once the User Stories have been identified, they are divided into smaller Tasks that describe the activities required for implementation. For this case study, a hypothetical development stack consisting of React.js for the frontend, FastAPI for the backend, and PostgreSQL for database management is considered.

The following implementation activities are considered for each selected User Story:

- **Frontend Development:** Implement the required user interface using React.js.
- **Backend Development:** Develop the required API endpoints and business logic using FastAPI.
- **Database Design:** Design the database schema and required relationships using PostgreSQL.
- **Unit Testing:** Develop and execute unit tests to validate the implemented functionality.
- **CI/CD Configuration:** Configure the required Continuous Integration and Continuous Deployment pipeline activities.

2.5.1 User Story 1: Track Packages

User Story:

> As a customer, I want to track my package so that I can know the current delivery status and estimated arrival of my order.

Tasks and Estimated Effort:

Task	Estimated Effort
Build the React UI	5 hours
Build the FastAPI Layer	6 hours
Build the Database Schema	8 hours
Write Unit Test Cases	4 hours
Configure CI/CD Pipeline	2 hours
Total Estimated Effort	**25 hours**

Table 2.1: Task Breakdown for Track Packages

The frontend provides the tracking interface, while the backend retrieves and processes the relevant shipment information. The database stores the necessary order and tracking information. Unit tests validate the implemented functionality, and the CI/CD pipeline supports automated build, testing, and deployment activities.

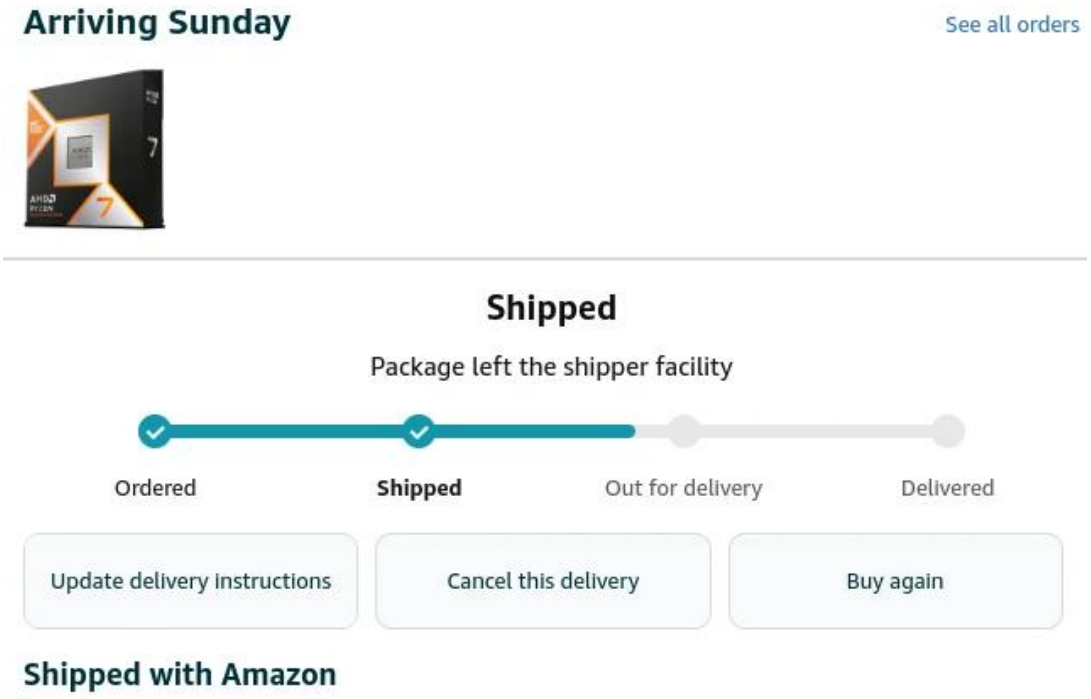


Figure 2.4: Package Tracking Interface on Amazon.in

2.5.2 User Story 2: Edit or Cancel Orders

User Story:

> As a customer, I want to edit or cancel an eligible order so that I can correct order details or cancel a purchase when required.

Tasks and Estimated Effort:

Task	Estimated Effort
Build the React UI	5 hours
Build the FastAPI Layer	6 hours
Build the Database Schema	8 hours
Write Unit Test Cases	4 hours
Configure CI/CD Pipeline	2 hours
Total Estimated Effort	**25 hours**

Table 2.2: Task Breakdown for Edit or Cancel Orders

The frontend allows customers to access available order modification options. The backend validates the order status and determines whether the requested modification or cancellation is permitted. The database maintains the relevant order information and modification records. Unit testing validates the different order modification scenarios.

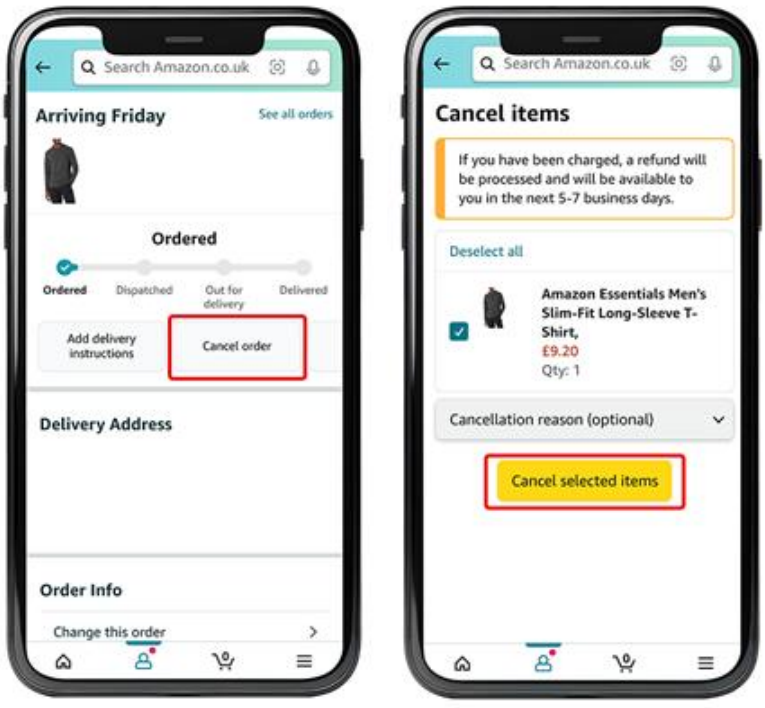


Figure 2.5: Order Modification and Cancellation Interface

2.5.3 User Story 3: Return and Exchange Items

User Story:

> As a customer, I want to return or exchange an eligible product so that I can obtain a replacement or refund when required.

The associated implementation Tasks include:

- Develop the return and exchange interface.
- Implement return and exchange API endpoints.
- Design database tables for return requests and their status.
- Write unit test cases for return eligibility and request processing.
- Configure the required CI/CD activities.

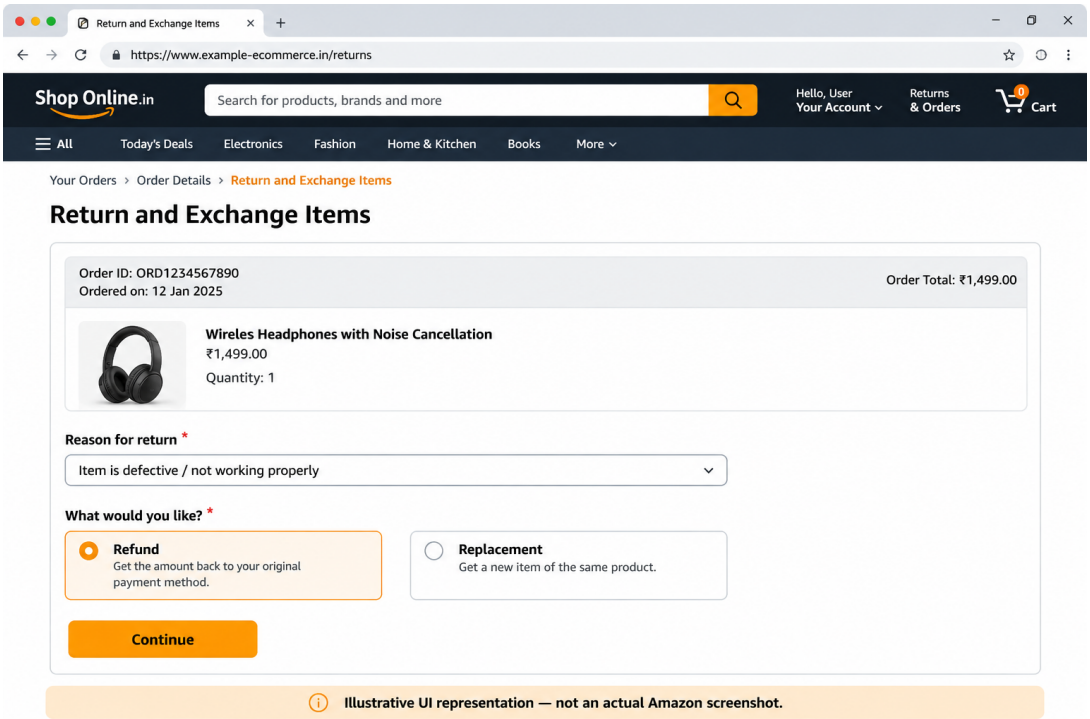


Figure 2.6: Return and Exchange Functionality

2.5.4 User Story 4: Print Labels

User Story:

> As a customer, I want to obtain and print the return shipping label so that I can send the product back through the applicable return process.

The associated implementation Tasks include:

- Develop the return label interface.
- Implement the backend functionality to generate or retrieve return labels.
- Store the required return shipment and label reference information.
- Write unit test cases for label generation and retrieval.
- Configure the required CI/CD activities.

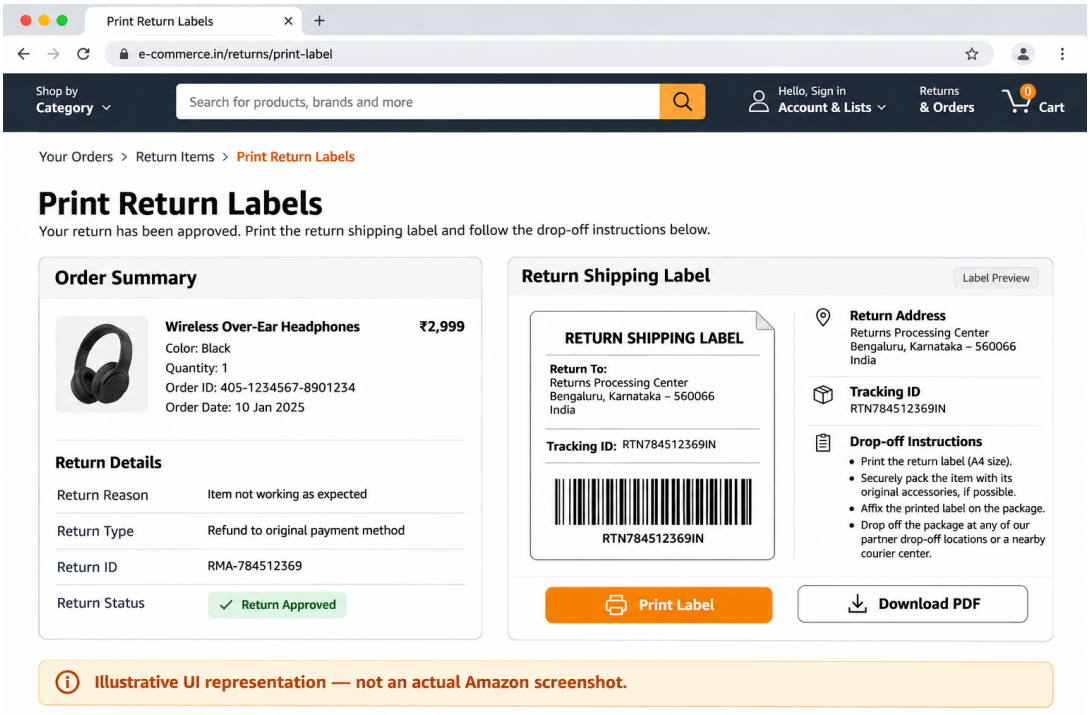


Figure 2.7: Return Shipping Label Functionality

2.6 Summary of the Work Item Hierarchy

The Customer Service Epic can be summarized using the following hierarchical structure.

- **Epic: Customer Service**

- **Feature: Your Orders**
  - User Story: Track Packages
  - User Story: Edit or Cancel Orders
- **Feature: Returns and Refunds**
  - User Story: Return and Exchange Items
  - User Story: Print Labels
- **Feature: Manage Address**
  - User Story: Add a Delivery Address
  - User Story: Edit a Delivery Address
  - User Story: Delete a Saved Address
  - User Story: Set a Default Address
- **Feature: Manage Prime**
  - User Story: View Membership Details
  - User Story: Manage Membership Renewal
  - User Story: View Membership Benefits

This hierarchy demonstrates how a large functional requirement can be decomposed into smaller Features, User Stories, and implementation Tasks. Such decomposition helps development teams understand requirements, distribute responsibilities, estimate effort, and monitor implementation progress.

Bibliography

[1] *Amazon*. URL: https://www.amazon.in/.

[2] *Software Engineering Tutorial*. URL: https://www.geeksforgeeks.org/software-engineering/.

[3] *What is Agile Project Management?* URL: https://www.atlassian.com/agile/project-management/.